CSC103-Programming Fundamentals

Lecture-4

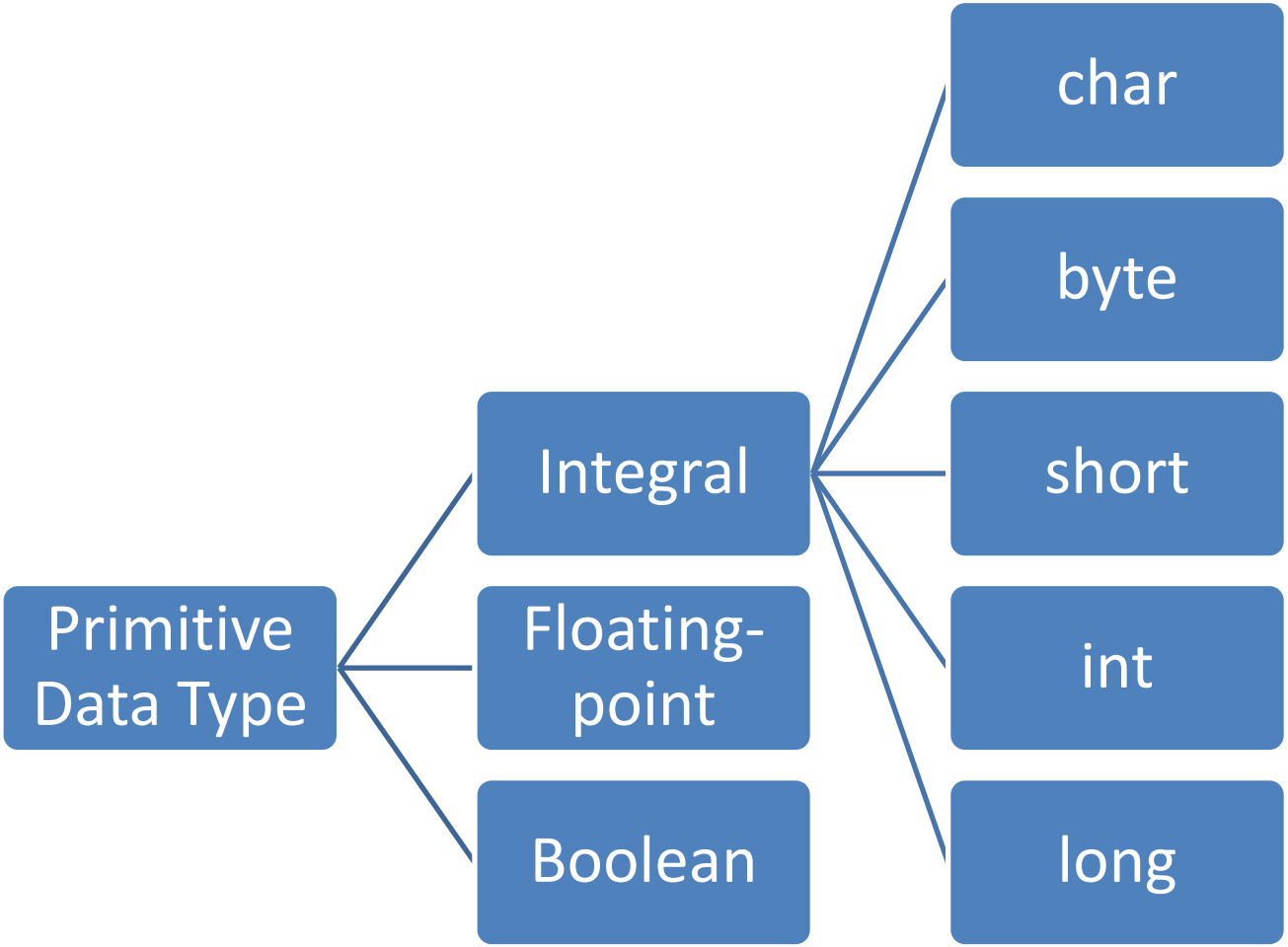
Instructor

Rizwan Rashid

Lecture Outline

- Java Primitive Data Types
- Literals – Numeric Literals
- Assignment Statement and Assignment Expression
- Augmented Assignment Operators
- Type Casting
- class String

Data Types


Primitive Data Type
Integral
Floating-point
Boolean
char
byte
short
int
long

Integral Data Types

Data Type	Values	Storage (in bytes)
`char`	0 to 65535 ($= 2^{16} - 1$)	2 (16 bits)
`byte`	-128 ($= -2^7$) to 127 ($= 2^7 - 1$)	1 (8 bits)
`short`	-32768 ($= -2^{15}$) to 32767 ($= 2^{15} - 1$)	2 (16 bits)
`int`	-2147483648 ($= -2^{31}$) to 2147483647 ($= 2^{31} - 1$)	4 (32 bits)
`long`	-9223372036854775808 ($= -2^{63}$) to 9223372036854775807 ($= 2^{63} - 1$)	8 (64 bits)

boolean Data Types

- Two values: true and false (called Logical values)

Floating-Point Data Types

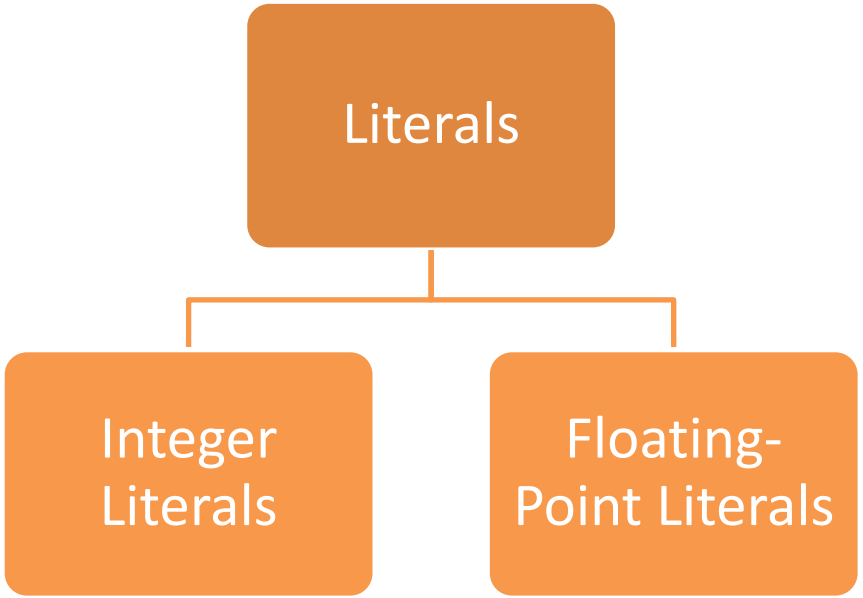
- To deal with decimal numbers

Type	Range	Storage	Precision
float	-3 .4E+38 and 3.4E+38	4 bytes	Single
double	-1.7E+308 and 1.7E+308	8 bytes	Double

Literals

- A *literal* is the source code representation of a fixed value
- Example

```
boolean result = true;
char capitalC = 'C';
byte b = 100;
short s = 10000;
int i = 100000;
```



Integer Literals

- An integer literal is assumed to be of the **`int`** type, whose value is between

  -2^{31}(-2147483648) and $2^{31} - 1$ (2147483647)

- Integer literal of the long type, append the letter L or l to it

```
int viewCount = 2147483647
```

```
long viewCount = 2147483648L
```

- Integer literals can be expressed by these number systems
  - Decimal
  - Hexadecimal
  - Binary
  - octal

```
int dec  = 26; //Decimal Number System
int hexa = 0X1A; //0x or 0X Number System
int bin  = 0b11010; //0b or 0B
int oct  = 032; //0
```

Floating-Point Literals

- By default, a floating-point literal is treated as a **double type** value
- Append the letter f or F to make it ***float***
- Append the letter d or D to make it ***double***
- ***Scientific Notation***
  - The floating point types (float and double) can also be expressed using **E or e: a * 10^b**

Using Underscore Characters in Numeric Literals

- In Java SE 7 and later , any number of underscore characters (_) can appear any where between digits in a numerical literal

```
long creditCardNumber = 1234_5678_9012_3456L;
long socialSecurityNumber = 999_99_9999L;
```

Assignment Statements and Assignment Expressions

An assignment statement designates a value for a variable and an assignment statement can be used as an expression in Java

- Syntax for Assignment Statement

```
variable = expression;
```

- An expression represents a computation involving values, variables, and operators that, taking them together, evaluates to a value.

```
int y = 1; // Assign 1 to variable y
int x = 5 * (3 / 2);        // Assign the value of the expression to x
x = y + 1; // Assign the addition of y and 1 to x
```

Numeric Operators

Name	Meaning	Example	Result
+	Addition	34 + 1	35
-	Subtraction	34.0 – 0.1	33.9
*	Multiplication	300 * 30	9000
/	Division	1.0 / 2.0	0.5
%	Remainder	20 % 3	2

Integer Division

- For Integer Division

5 / 2 yields an integer 2

- To perform a float-point division, one of the operands must be a floating-point number

5.0 / 2 yields a double value 2.5

- For Remainder

5 % 2 yields 1

Arithmetic Expressions

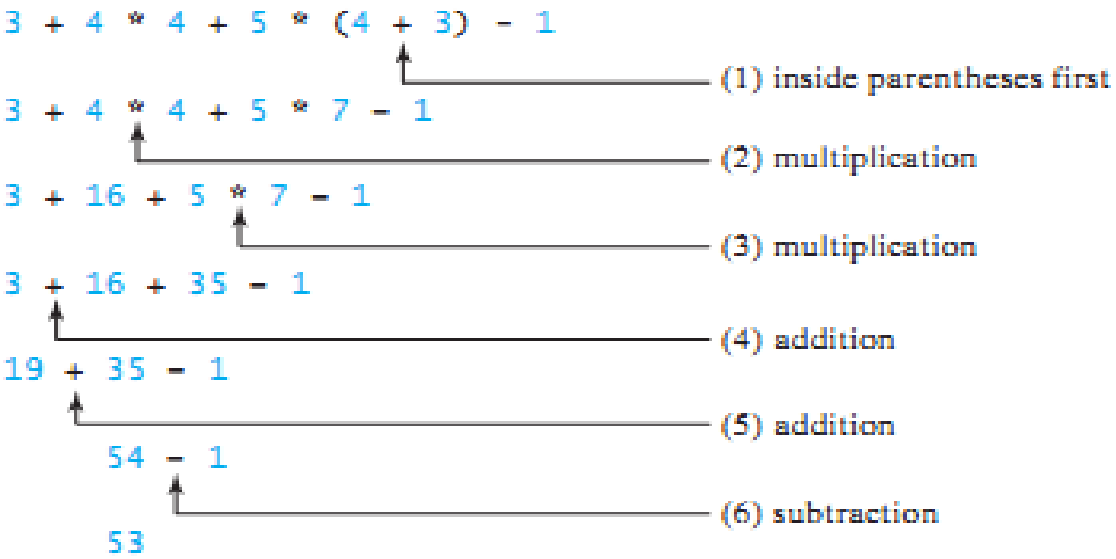
$$\frac{3+4x}{5} - \frac{10(y-5)(a+b+c)}{x} + 9\left(\frac{4}{x} + \frac{9+x}{y}\right)$$

is translated to

```
(3+4*x)/5 – 10*(y-5)*(a+b+c)/x + 9*(4/x + (9+x)/y)
```

How to Evaluate an Expression

- You can safely apply the arithmetic rule for evaluating a Java expression
- *,/,%
- +,-



Augmented Assignment Operators

The operators +, -, *, /, and % can be combined with the assignment operator to form augmented operators.

Operator	*Name*	*Example*	*Equivalent*
+=	Addition assignment	`i += 8`	`i = i + 8`
-=	Subtraction assignment	`i -= 8`	`i = i - 8`
*=	Multiplication assignment	`i *= 8`	`i = i * 8`
/=	Division assignment	`i /= 8`	`i = i / 8`
%=	Remainder assignment	`i %= 8`	`i = i % 8`

Example

$$x \mathrel{/}= 4 + 5.5 * 1.5;$$

is same as

$$x = x / (4 + 5.5 * 1.5);$$

Numeric Type Conversions

- **Implicit Casting**: Done automatically by Java (mixed expression)
- **Explicit Casting**: Done by programmer

(dataTypeName) expression

Expression	Evaluates to
`(int) (7.9)`	`7`
`(int) (3.3)`	`3`
`(double) (25)`	`25.0`
`(double) (5 + 3)`	`= (double) (8) = 8.0`
`(double) (15) / 2`	`= 15.0 / 2` (because `(double) (15) = 15.0`) `= 15.0 / 2.0 = 7.5`

class String

- *A string is a sequence of characters.*
- The **char** type represents only one character
- String Type is use to represent collection of characters

  String message = "Welcome to Java";

- **class String** is a predefined class in the Java library (java.lang)
- The String type is not a primitive type

class String

- A string that contains no characters is called a *null* or *empty string*
- Examples
  - “William Jacob”
  - “Mickey”
  - “”
- Syntax

String variableName = “Some String”;

Character String
String literal
String constant